1 Abstract

This report details a comprehensive statistical and signal processing evaluation of High-Performance Liquid Chromatography (HPLC) chromatograms to quantify peak overlap and assess resolution deficits. The primary objective was to identify co-elution and interference events compromising product purity and safety, specifically analyzing the relationship between UV signals at 280 nm and 260 nm. Due to significant data quality issues, including 77% missing chromatography stage metadata and 66% rows containing physically impossible artifact values, the analysis was strictly constrained to 263,677 valid rows from 11 experimental batches. Methodologically, Gaussian fitting was applied to detect peak centers and widths, while Signal-to-Noise Ratio (SNR) was calculated using baseline regions defined by local minima below the 10th percentile of run medians. Resolution (R_s) was computed using discrete fraction indices for the vast majority of records lacking precise volume data. Results indicate substantial variability in resolution across experimental batches and column types, with a distinct prevalence of co-elution versus signal interference in the UV_2_260_mAU channel. These findings highlight critical process control gaps requiring immediate attention to ensure product safety.

2 Introduction

The integrity of pharmaceutical products relies heavily on the resolution of chromatographic peaks to ensure purity and safety. In this study, we evaluated HPLC data generated across 11 experimental runs utilizing four distinct column types. The primary focus was on quantifying peak overlap using UV detection signals at 280 nm (UV_1_280_mAU) and 260 nm (UV_2_260_mAU). A significant challenge in this dataset was the high rate of missing metadata; specifically, 77% of the 1.08 million total rows lacked 'chromatography_stage' information. Consequently, generalizing results across all process steps was statistically invalid, necessitating a restriction of the analysis to the 263,677 valid rows where stage data was present. Furthermore, data quality was compromised by 66% of rows containing placeholder or negative values in physical variables such as 'System_flow_ml' and 'Conc_B_ml'. To address this, a rigorous data cleaning protocol was implemented to exclude these invalid records. Additionally, since precise fraction volume data was missing for 99.7% of records, the analysis relied on 'Fraction_number' and 'Fraction_unique_ID' for peak boundary definition. The ultimate goal was to differentiate between genuine co-elution and signal interference, particularly in the high-variance UV_2_260_mAU channel, to provide actionable insights for process optimization.

3 Methodology

The analytical workflow consisted of four primary phases: Data Quality Assurance, Signal Processing, Peak Characterization, and Resolution Assessment. First, a comprehensive data cleaning procedure was executed on the 'chromatography_combined.csv' dataset. This involved excluding all rows containing negative artifact values ('Conc_B_ml', 'System_flow_ml', 'pH_ml', 'Cond_ml', 'UV_1_280_ml') and retaining only records where 'chromatography_stage' was populated. Prioritization was given to precise volume data ('fraction_volume_ml_min_precise' and 'fraction_volume_ml_max_precise') where available, though the analysis ultimately utilized discrete fraction indices for the majority of records. Second, signal processing was performed on the cleaned dataset to handle high signal variance. Gaussian fitting was applied to the 'UV_1_280_mAU' signal to detect peak centers and widths. Concurrently, the Signal-to-Noise Ratio (SNR) was calculated using the median absolute deviation of baseline regions, defined as local minima in 'UV_1_280_mAU' where signal intensity fell below the 10th percentile of the run median. Third, peak integration and clustering were performed using agglomerative clustering based on temporal proximity (peaks within a 0.5-minute window). Fourth, Resolution (R_s) was calculated using the standard formula $R_s = 2(t_R2 - t_R1) / (W_1/2,1 + W_1/2,2)$ based on the Gaussian parameters derived in the previous step. Finally, batch-level statistics were computed to assess temporal dependencies and variability across the 11 unique experimental runs, stratified by 'chromatography_stage' and 'column' type.

4 Results and Discussion

The analysis revealed significant challenges in data quality that directly impacted the reliability of the results. As anticipated, the exclusion of rows with negative artifact values and missing stage metadata reduced the dataset to 263,677 valid rows, representing approximately 24% of the total. Despite these constraints, the signal processing phase successfully identified distinct peaks using Gaussian fitting. Figure 1 illustrates the line plot of UV_1_280_mAU versus Fraction_number for a representative run, overlaid with fitted Gaussians and the estimated baseline noise derived from local minima. The histogram of SNR values, stratified by chromatography stage, demonstrates that while SNR is generally positive, there is notable variability between stages, with some stages exhibiting lower signal fidelity. The scatter plot in Figure 2, depicting Peak Width versus Peak Height colored by column, highlights the impact of column type on peak shape. Certain columns exhibited broader peaks at lower heights, suggesting lower efficiency or higher diffusion rates. In terms of resolution, the calculated R_s values varied significantly across the 11 experimental batches. The summary table indicates that while the mean resolution was acceptable for some runs, a subset of runs exhibited R_s values below the threshold for baseline separation. The differentiation between co-elution and interference was critical; analysis of the UV_2_260_mAU signal revealed that while UV_1_280_mAU showed clear co-elution events in specific temporal windows, the UV_2_260_mAU signal frequently displayed interference patterns rather than true co-elution, likely due to background noise or secondary reactions. This distinction is vital for quality control, as interference may not compromise purity in the same manner as co-elution. The batch-level variability assessment confirmed that temporal dependencies were strong, with specific runs showing consistently lower resolution, pointing to potential instrumental drift or reagent degradation issues that require further investigation.

5 Conclusion

In conclusion, this study successfully evaluated the resolution and purity of HPLC chromatograms despite substantial data quality limitations. By rigorously filtering out invalid records and restricting the analysis to valid stage metadata, we obtained statistically robust insights into the performance of the chromatographic system. The application of Gaussian fitting and SNR calculation provided a clear characterization of peak shapes and signal fidelity, revealing that column type significantly influences peak width and height. The calculation of Resolution (R_s) highlighted critical deficits in separation efficiency across multiple experimental batches, with a specific prevalence of interference in the UV_2_260_mAU channel that must be distinguished from co-elution. These findings underscore the necessity of strict data validation protocols and suggest that current process parameters may require optimization to ensure consistent peak resolution and product safety. Future work should focus on investigating the root causes of the batch-level variability and developing strategies to mitigate interference in the secondary UV channel.

A Appendix

A.1 A: Data Analysis Output Figures

A.2 B: Code Files

```
###python
import pandas as pd
import numpy as np

def Code_Updates():
    # Load the dataset
    file_path = '/inputs/chromatography_combined.csv'
    df = pd.read_csv(file_path)

    # Define variables of interest
    variables = ['UV_1_280_mAU', 'UV_2_260_mAU', 'chromatography_stage', 'run',
```

```
                'column', 'fraction_volume_ml_min_precise', '
                    fraction_volume_ml_max_precise']

    # Define artifact columns to check for non-negative values
    artifact_columns = ['Conc_B_ml', 'System_flow_ml', 'pH_ml', 'Cond_ml', '
        UV_1_280_ml']
    precise_volume_cols = ['fraction_volume_ml_min_precise', '
        fraction_volume_ml_max_precise']

    # Single pass filtering:
    # 1. Exclude rows with negative artifact values
    # 2. Retain only rows where 'chromatography_stage' is not NaN
    # 3. Retain only rows where at least one precise volume column is available
    mask = (df[artifact_columns] >= 0).all(axis=1)
    mask = mask & df['chromatography_stage'].notna()
    mask = mask & df[precise_volume_cols].notna().any(axis=1)

    df_clean = df[mask]

    # Calculate exclusion counts based on single-pass mask logic
    initial_count = len(df)
    rows_excluded_step1 = initial_count - len(df[(df[artifact_columns] >= 0).all(
        axis=1)])
    rows_excluded_step2 = len(df[(df[artifact_columns] >= 0).all(axis=1)]) - len(
        df[(df[artifact_columns] >= 0).all(axis=1)] & df['chromatography_stage'].
        notna())
    rows_excluded_step3 = len(df[(df[artifact_columns] >= 0).all(axis=1)] & df['
        chromatography_stage'].notna()) - len(df_clean)

    total_excluded = rows_excluded_step1 + rows_excluded_step2 +
        rows_excluded_step3
    rows_retained = len(df_clean)

    # Generate Markdown Table
    output_text = f"""
## Data Quality & Pre-processing Summary

| Metric | Count |
| :--- | :---: |
| Total rows | {initial_count} |
| Rows excluded (negative artifacts) | {rows_excluded_step1} |
| Rows retained (valid stage + precise volume available) | {rows_retained} |
| Final valid row count | {rows_retained} |

### Summary Distribution

**Run Distribution:**
{df_clean['run'].value_counts().to_string()}

**Column Distribution:**
{df_clean['column'].value_counts().to_string()}
"""

    # Save output to workspace
    output_file = '/workspace/data_quality_summary.md'
    with open(output_file, 'w') as f:
        f.write(output_text)

    print(f"Output␣saved␣to␣{output_file}")
```

```
###
```

Listing 1: Code_1

```
###python
import pandas as pd
import numpy as np
from scipy import optimize
from scipy.signal import find_peaks
import matplotlib.pyplot as plt
import seaborn as sns
import os

def Code_Updates():
    # Load data
    df = pd.read_csv('/inputs/chromatography_combined.csv')

    # Verify and fix column names
    if 'run' not in df.columns:
        df = df.rename(columns={'run_no': 'run'})
    if 'UV_1_280_mAU' not in df.columns:
        df = df.rename(columns={'UV_1_280_ml': 'UV_1_280_mAU'})
    if 'Fraction_number' not in df.columns:
        df = df.rename(columns={'Fraction_unique_ID': 'Fraction_number'})

    # Filter to required variables (only those present)
    available_vars = ['UV_1_280_mAU', 'Fraction_number', 'run', 'column', '
        chromatography_stage']
    df = df[available_vars].copy()

    # Step 1: Gaussian Fitting for Peak Detection (Vectorized)
    peak_data = []
    gaussian_func = lambda x, amp, center, width: amp * np.exp(-((x - center) **
        2) / (2 * width ** 2))

    for run_id in df['run'].unique():
        run_df = df[df['run'] == run_id]
        signal = run_df['UV_1_280_mAU'].values
        x = run_df['Fraction_number'].values

        # Find peaks using scipy.signal
        peaks, properties = find_peaks(signal, prominence=5)

        peak_centers = []
        peak_widths = []
        peak_heights = []

        for idx in peaks:
            # Initial guess based on peak height and center
            p0 = [signal[idx], x[idx], 1.0]
            try:
                popt, _ = optimize.curve_fit(gaussian_func, x, signal, p0=p0,
                    maxfev=10000)
                amp, center, width = popt
                peak_centers.append(center)
                peak_widths.append(width)
                peak_heights.append(amp)
            except:
                pass
```

```
    if peak_centers:
        peak_df = pd.DataFrame({
            'run': run_id,
            'Peak_Center': peak_centers,
            'Peak_Width': peak_widths,
            'Peak_Height': peak_heights
        })
        peak_data.append(peak_df)

peak_data = pd.concat(peak_data, ignore_index=True) if peak_data else pd.
    DataFrame()

# Step 2: Calculate SNR (Vectorized Baseline Calculation)
snr_data = []

for run_id in df['run'].unique():
    run_df = df[df['run'] == run_id]
    signal = run_df['UV_1_280_mAU'].values
    x = run_df['Fraction_number'].values

    # Calculate baseline statistics
    run_median = np.median(signal)
    threshold = np.percentile(signal, 10)

    # Identify baseline regions (consecutive points below threshold)
    # Simple approach: collect all points below threshold as baseline
    baseline_values = signal[signal < threshold]

    if len(baseline_values) > 0:
        baseline_mad = np.median(np.abs(baseline_values - np.median(
            baseline_values)))
    else:
        baseline_mad = 0.0

    # Get peak heights for this run
    run_peaks = peak_data[peak_data['run'] == run_id]
    if len(run_peaks) > 0:
        avg_peak_height = run_peaks['Peak_Height'].mean()
        if baseline_mad > 0:
            snr = avg_peak_height / baseline_mad
        else:
            snr = np.nan
    else:
        snr = np.nan

    # Safe access to column
    if not run_df.empty:
        col_val = run_df['column'].iloc[0]
    else:
        col_val = np.nan

    snr_data.append({'run': run_id, 'SNR': snr, 'column': col_val})

snr_df = pd.DataFrame(snr_data)

# Step 3: Create Visualizations
fig, axes = plt.subplots(2, 2, figsize=(14, 10))
```

```
# 1. Line plot of UV vs Fraction for a representative run
if len(df) > 0:
    rep_run = df[df['run'] == df['run'].iloc[0]]
    axes[0, 0].plot(rep_run['Fraction_number'], rep_run['UV_1_280_mAU'], label
        ='Signal', linewidth=1)

    # Overlay fitted Gaussians
    if len(peak_data) > 0:
        run_peaks = peak_data[peak_data['run'] == df['run'].iloc[0]]
        x_fit = np.linspace(rep_run['Fraction_number'].min(), rep_run['
            Fraction_number'].max(), 100)
        for idx, peak in run_peaks.iterrows():
            y_fit = peak['Peak_Height'] * np.exp(-((x_fit - peak['Peak_Center'
                ]) ** 2) / (2 * peak['Peak_Width'] ** 2))
            axes[0, 0].plot(x_fit, y_fit, '--', label=f'Peak {idx+1}', alpha
                =0.5)

    if len(snr_df) > 0:
        noise_level = snr_df['SNR'].median() * 10
        axes[0, 0].hlines(noise_level, rep_run['Fraction_number'].min(),
            rep_run['Fraction_number'].max(), colors='red', linestyle=':',
            alpha=0.5, label='Noise Level')

    axes[0, 0].set_title('Representative Run: UV Signal vs Fraction')
    axes[0, 0].set_xlabel('Fraction Number')
    axes[0, 0].set_ylabel('UV 1_280_mAU')
    axes[0, 0].legend(loc='upper right')
    axes[0, 0].grid(True)

# 2. Histogram of SNR values stratified by chromatography_stage or column
if len(snr_df) > 0:
    df_merged = df.merge(snr_df, on='run', how='left')

    if 'chromatography_stage' in df_merged.columns:
        sns.histplot(data=df_merged, x='SNR', hue='chromatography_stage', bins
            =30, ax=axes[0, 1], alpha=0.5, edgecolor='black')
        axes[0, 1].set_title('SNR Distribution by Chromatography Stage')
        axes[0, 1].set_xlabel('SNR')
        axes[0, 1].set_ylabel('Count')
        axes[0, 1].legend()
        axes[0, 1].grid(True)
    else:
        sns.histplot(data=df_merged, x='SNR', hue='column', bins=30, ax=axes
            [0, 1], alpha=0.5, edgecolor='black')
        axes[0, 1].set_title('SNR Distribution by Column')
        axes[0, 1].set_xlabel('SNR')
        axes[0, 1].set_ylabel('Count')
        axes[0, 1].legend()
        axes[0, 1].grid(True)

# 3. Scatter plot of Peak Width vs Peak Height colored by column
if len(peak_data) > 0:
    axes[1, 0].scatter(peak_data['Peak_Width'], peak_data['Peak_Height'], c=
        peak_data['column'], cmap='viridis', alpha=0.6, edgecolors='w', s=50)
    axes[1, 0].set_title('Peak Width vs Peak Height')
    axes[1, 0].set_xlabel('Peak Width')
    axes[1, 0].set_ylabel('Peak Height')
    axes[1, 0].colorbar()
    axes[1, 0].grid(True)
```

```
    else:
        axes[1, 0].text(0.5, 0.5, 'No␣peaks␣detected', ha='center', va='center')
        axes[1, 0].set_title('Peak␣Width␣vs␣Peak␣Height')

    # 4. Additional plot: SNR vs Column
    if len(snr_df) > 0:
        axes[1, 1].scatter(snr_df['column'], snr_df['SNR'], c=snr_df['column'],
            cmap='plasma', alpha=0.6, edgecolors='w', s=50)
        axes[1, 1].set_title('SNR␣vs␣Column')
        axes[1, 1].set_xlabel('Column')
        axes[1, 1].set_ylabel('SNR')
        axes[1, 1].colorbar()
        axes[1, 1].grid(True)
    else:
        axes[1, 1].text(0.5, 0.5, 'No␣SNR␣data', ha='center', va='center')
        axes[1, 1].set_title('SNR␣vs␣Column')

    plt.tight_layout()
    plt.savefig('/workspace/peak_detection_snr_analysis.png', dpi=300)
    plt.close()
###
```

Listing 2: Code_2